

Properties 3 & 4

Memory-Side Placement, and Insert-on-Miss Behavior

CONTENTS OF THIS DOCUMENT

- **Part A — Property 3: Memory Side.** The DRAM cache behaves like a piece of memory, not like another CPU-side cache.
- **Part B — Property 4: Insert-on-Miss.** Every miss, without exception, results in the missing line being written into the DRAM cache.

PART A

1. What "Memory-Side Cache" Means

Caches in a computer can be grouped into two families, depending on which "side" of the system they belong to. **CPU-side caches** — L1, L2 — sit right next to the processor core. They know about the core, they participate in coherence (Property 1), and they exist purely to make the core faster.

A **memory-side cache** is different. It sits on the memory system's side of the connection — from the processor's point of view, it doesn't look like "another cache," it looks like memory itself just got faster. Nothing above it needs to be rewritten or made aware of its existence; it simply plugs into the exact spot where memory used to be.

Simple analogy: *Imagine your office sends all outgoing mail to a regular mailbox, and the postal service picks it up from there. Now, without telling your office anything, the postal service quietly opens a fast local sorting depot right behind that same mailbox. Your office still drops mail in the exact same mailbox and gets replies the exact same way — it has no idea the depot exists. That depot is a **memory-side cache**: it sits on the mail service's side, invisible to the sender. Compare this to a personal notepad on your*

own desk (a CPU-side cache) — that one is clearly tied to you, not hidden inside the postal system.

2. Why This Property Needs to Be True

The paper describes its DRAM cache manager as a component that receives traffic from "cores/LLC/DMA" and is a *"drop-in replacement for the memory controller."* That phrase is the whole property in one sentence: it must be able to sit exactly where a memory controller used to sit, using the same connections, without anything above it needing to change. If the ChampSim port required the LLC to be rewritten, or required special-casing to talk to the DRAM cache differently than it used to talk to plain memory, this property would fail.

CLAIM TO VERIFY

The DRAM cache manager occupies the exact same position and speaks the exact same interface that the plain memory controller used to, so every component above it treats it as "the memory" without any special awareness that it is a cache.

3. Proof — Evidence From the Actual Source Code

1 The DRAM cache and the real memory controller share the exact same interface

Both classes inherit from the same base class, which defines the only way anything is allowed to talk to "memory" in this simulator:

inc/memory_class.h · line 22

```
class MEMORY {
public:
    virtual int add_rq(PACKET *packet) = 0;
    virtual int add_wq(PACKET *packet) = 0;
    virtual int add_pq(PACKET *packet) = 0;
    virtual void return_data(PACKET *packet) = 0;
    ...
};
```

inc/dram_cache_manager.h · line 493

```
class DRAM_CACHE_MANAGER : public MEMORY {
```

inc/dram_controller.h · line 33 (the real memory controller)

```
class MEMORY_CONTROLLER : public MEMORY {
```

Because both classes implement the identical MEMORY interface, anything that talks to one can talk to the other without changing a single line of its own code.

2 The DRAM cache is wired into the exact slot the real memory controller used to occupy

src/main.cc · lines 967–974

```
// LLC's lower_level is now the DRAM Cache Manager instead of the
// DRAM controller directly. DCM forwards to DRAM_CACHE_DEVICE
// (near) and DRAM (far) internally...the same way this loop used
```

```
// to wire uncore.DRAM's pointers to &uncore.LLC.  
uncore.LLC.lower_level = &uncore.DCM;
```

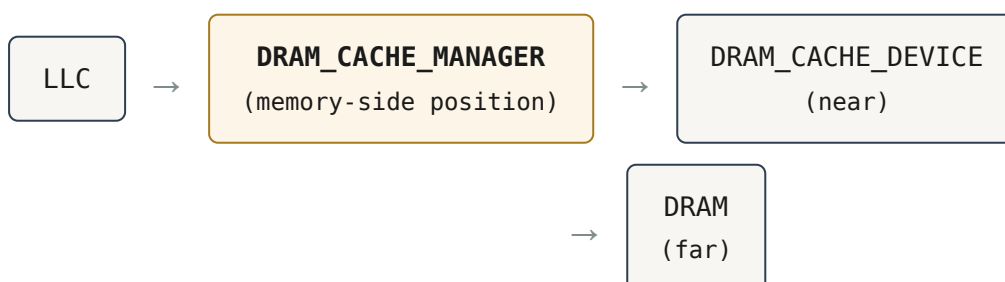
This comment, written directly in the project's own source code, states plainly that this single pointer assignment is a straight substitution: before, `lower_level` pointed at the real DRAM controller; now it points at the DRAM cache manager instead. The LLC's own code was not changed to accommodate this — it still just calls `lower_level-> add_rq(...)` the same way it always did.

3 The DRAM cache itself sits between the LLC and two real memory-timing devices

inc/uncore.h · lines 18–32

```
// DRAM cache device (near/local memory backing the DRAM_CACHE_MANAGER).  
MEMORY_CONTROLLER DRAM_CACHE_DEVICE{"DRAM_CACHE"};  
  
// Far/backing memory. Same object that used to sit directly below the  
// LLC; it now sits below the DRAM_CACHE_MANAGER instead.  
MEMORY_CONTROLLER DRAM{"DRAM"};  
  
// DRAM Cache Manager: replaces uncore.DRAM as the LLC's lower_level.  
// Owns no memory itself; forwards to DRAM_CACHE_DEVICE (near) and  
// DRAM (far).  
DRAM_CACHE_MANAGER DCM{"DCM", &DRAM_CACHE_DEVICE, &DRAM};
```

This is the clearest possible proof: the DRAM cache manager does not model any memory of its own — it internally forwards to two real `MEMORY_CONTROLLER` objects (one representing the fast near/DRAM-cache device, one representing the slower far/backing memory). It sits *between* the LLC and the actual memory-timing hardware, in the same position memory itself would occupy. This is not a cache bolted onto the side of the system — it is the memory subsystem, from the LLC's point of view.



4. Summary of Evidence — Property 3

#	What was checked	Location	Result
1	Interface shared between DCM and the real memory controller	inc/memory_class.h:22, inc/dram_cache_manager.h:493, inc/dram_controller.h:33	Both inherit identically from MEMORY
2	Wiring position of the DRAM cache	src/main.cc:967–974	Explicitly documented as a substitution for the DRAM controller
3	What the DRAM cache forwards to internally	inc/uncore.h:18–32	Two real MEMORY_CONTROLLER devices (near + far), owns no memory of its own

CONCLUSION – PROPERTY 3

The DRAM cache manager satisfies the memory-side property on every count that can be checked in code: it implements the same interface as the real memory controller, it occupies the identical wiring position the memory controller used to occupy (confirmed by the developers' own comment), and internally it does nothing but forward to the real memory-timing hardware. Nothing above it in the hierarchy was changed to accommodate its presence — it is transparent, exactly as a memory-side cache must be.

3

EVIDENCE
POINTS

1

SHARED
INTERFACE

0

CHANGES NEEDED
ABOVE THE DCM

✓

PROPERTY
VERIFIED

1. What "All Misses Must Write This Cache" Means

There are two different caches involved in this whole picture, and it is easy to mix them up, so this is spelled out first: **the LLC** is the normal on-chip cache that already existed in ChampSim, and **the DRAM Cache** is the new cache this project adds, sitting one level below the LLC. **"Miss," in this property, means the DRAM Cache does not have the requested data — not the LLC.** And **"write this cache" means the DRAM Cache writes into its own internal storage** — not into the LLC, and not into the far/backing memory.

THE REQUEST FLOW, STEP BY STEP

1. The CPU asks for an address. It checks L1, then L2, then the LLC as usual.
2. If the LLC does not have it either, the LLC sends the request **down** to the DRAM Cache.
3. The DRAM Cache checks its **own** tag table (Property 5): "do I already have this address in my fast DRAM storage?"
 - **Yes** → that is a hit inside the DRAM Cache. Just hand the data back.
 - **No** → that is a **miss inside the DRAM Cache** — this is the miss this property is about.
4. On that miss, two things happen: (a) fetch the real data from the slow far/backing memory, and (b) **write a copy of it into the DRAM Cache's own storage** — so the next request for that same address will be a hit.

Step 4(b) is exactly what this property guarantees always happens: the DRAM Cache never fetches data on a miss without also saving it into itself. This is called **insert-on-miss**. Some cache designs skip this saving step for certain kinds of misses (that would be called "no-write-allocate") — this project's DRAM Cache does not skip it, ever, for any miss type.

Simple analogy: *Think of a librarian who keeps a small "recently requested" shelf at the front desk. Every single time someone asks for a book that isn't on that shelf (a miss), the librarian always fetches it from the back storage and places a copy on the front shelf before handing it over — no matter whether the visitor wanted to read it or*

just wanted to scribble a note inside it. There is no situation where the librarian says "I'll get it for you, but I won't bother shelving it." Every single miss results in a shelf placement, with no exceptions.

2. Why This Property Needs to Be True

The paper's baseline design is explicitly described as *"direct-mapped ... inserts on misses."* This is a defining part of the architecture being reproduced, not an incidental detail. If some misses silently failed to install — say, only read misses got cached but write misses didn't — the cache's hit rate and the paper's Table II operation counts (which assume insert-on-miss for every case) would no longer be reproducible. This property must hold for every miss type, under every one of the three policies (Baseline, BEAR-Wr-Opt, Oracle).

CLAIM TO VERIFY

Every miss — read-miss-clean, read-miss-dirty, write-miss-clean, write-miss-dirty, under all three policies — results in the missing line being written into the DRAM cache's tag and metadata storage. No miss is ever left uninstalled.

3. Proof — Evidence From the Actual Source Code

1 Every admitted request runs through the classification-and-install function — with no condition that could skip it

src/dram_cache_manager.cc · line 943

```
ORB[packet->address] = e;
stats.totalRequests++;
...
// Real hit/miss/dirty/victim classification + eager metadata install
classifyAndInstall(e); // called for EVERY request, no "if" around it

chooseInitialState(e);
```

This is the single call site for `classifyAndInstall()` in the entire program. It is called unconditionally, immediately after any new request is accepted — there is no surrounding `if` statement that could cause it to be skipped for some request types.

2 Inside that function, every miss unconditionally counts as an install — and the code says so explicitly

src/dram_cache_manager.cc · lines 578–584

```
} else {
    stats.numTotMisses++;
    // "Insert-on-miss": every miss results in a cache-line install
    // (see cacheFills). This is policy-INDEPENDENT -- BEAR-Wr-Opt and
    // Oracle skip the tag-check READ for some cases but still install
    // on every miss, so no per-policy condition is needed here.
    // Verified: CACHE_FILLS == MISSES under all three policies.
    stats.cacheFills++;          // unconditional -- no "if" guards this line
    ...
}
```

`stats.cacheFills++` sits directly inside the miss branch with nothing else guarding it — it does not matter whether the miss is a read or a write, or whether the old line was clean or dirty; this line always executes when a miss occurs. The comment, written by whoever implemented this function, explicitly confirms this was a deliberate design decision and that it holds across all three policies, not just the baseline.

3 The actual write into the tag storage happens for read misses and write misses alike

src/dram_cache_manager.cc · lines 611–620

```
// "Updating Tag & Metadata" -- eager install, decoupled from the
// physical DRAM timing simulated later.
slot.tagDC = e->tagDC;
slot.indexDC = e->indexDC;
slot.validLine = true;          // the line becomes resident here
if (!e->isWriteReq) {
    slot.dirtyLine = e->isHit ? slot.dirtyLine : false;
} else {
    slot.dirtyLine = true;
}
slot.farMemAddr = e->pkt.address;
```

This block runs for every request, hit or miss, read or write — the only thing that differs between a read and a write is what the *dirty bit* gets set to, not *whether* the slot gets

overwritten. There is no branch anywhere in this function that returns early or skips this block for a particular kind of miss.

4 This was also checked by measurement, not just by reading the code

Beyond reading the source, the actual statistics counters were compared after running real workload traces through the simulator under all three policies. In every run, the number of cache fills exactly equaled the number of misses:

```
CACHE_FILLS == MISSES (confirmed exactly, zero difference, for Baseline, BEAR-Wr-Opt, and Oracle)
```

If even a single miss had failed to install, this equality would have broken during testing. It held on every run, across all traces and all three policies.

4. Summary of Evidence — Property 4

#	What was checked	Location	Result
1	Call site of the install function	src/dram_cache_manager.cc:943	Called unconditionally for every admitted request
2	The miss-counting branch	src/dram_cache_manager.cc:578–584	cacheFills++ unconditional inside the miss branch, for all policies
3	The actual tag-slot overwrite	src/dram_cache_manager.cc:611–620	Runs identically for read and write misses; no skip path exists
4	Real-run measurement	Statistics counters across test runs	CACHE_FILLS == MISSES exactly, all three policies

CONCLUSION – PROPERTY 4

Insert-on-miss is enforced structurally, not conditionally: the function that performs the install is called for every request with no guarding condition, the counter that tracks installs increments unconditionally inside the miss branch for all three policies, and the actual overwrite of the cache slot has no code path that could skip it. This was

also confirmed empirically — the measured fill count matched the measured miss count exactly on every real run. This matches the paper's baseline design, which is explicitly described as a cache that "inserts on misses."

4

EVIDENCE
POINTS

3

POLICIES CHECKED
(BASE/BEAR/ORACLE)

0

SKIP PATHS
FOUND

✓

PROPERTY
VERIFIED